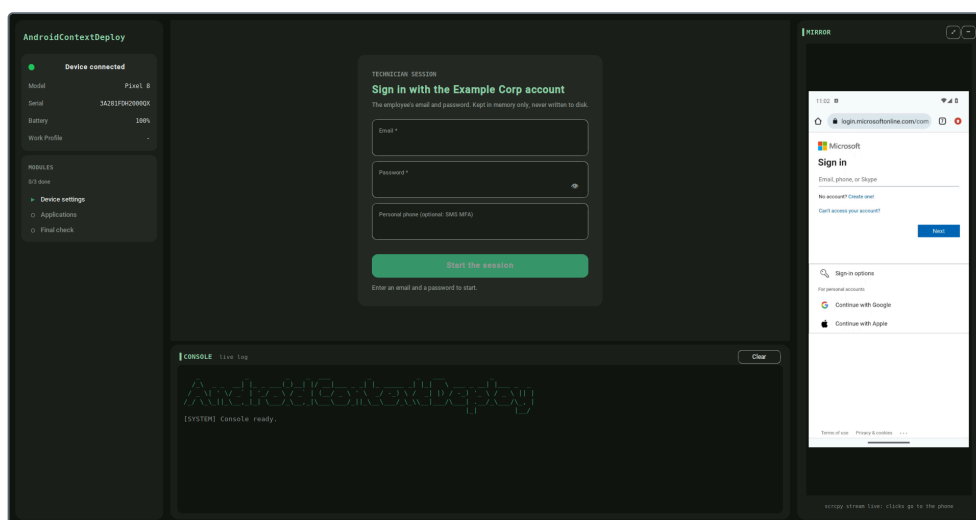


AndroidContextDeploy

Phone preparation, Intune enrollment and quick device diagnostics

Technician's manual

What the tool does, what it leaves to you,
and how to extend it



Contents

1	Introduction	1
1.1	Why this tool exists	1
1.2	Installing and launching it	1
1.3	What this manual covers	2
2	Quick start and every feature	3
2.1	Before you run it	3
2.2	Connecting the phone	3
2.3	The window at a glance	3
2.4	Starting a Session	4
2.5	The Modules	4
3	What stays manual	10
4	How the project is put together	11
4.1	Context — who talks to whom	11
4.2	Containers — the pieces inside	11
4.3	The folder tree	11
4.4	Sequence — one app through the Applications Module	12
5	The technologies, explained simply	13
5.1	ADB — the cable that talks	13
5.2	uiautomator — reading a screen without looking at it	13
5.3	Android Enterprise and the Work Profile	13
5.4	scrcpy, H.264 and PyAV — the Mirror	13
5.5	CustomTkinter, threads and queues — the window	14
5.6	JSON — the manifest and the languages	14
5.7	PyInstaller — from Python to an executable	14
5.8	pytest and fakes — tests without a phone	14
6	Enrollment in depth	16
6.1	What Enrollment is	16
6.2	The gate: the managed Play Store, never a timer	16
6.3	The named screens	16
6.4	Why the Organization matters	17
7	Extending it	18
7.1	First: does it need code at all?	18
7.2	Adding a UI Language	18
7.3	Teaching it a new screen	18
7.4	Adding a Module	19
7.5	Building a release	20
8	Troubleshooting	21
8.1	Reading the Checklist	21
8.2	Common problems	21
8.3	Getting more detail	21

Introduction

1.1 Why this tool exists

A company phone straight out of the box is not a finished phone. Before an employee can use it, somebody has to enroll it with the Company Portal, let Intune create the Work Profile, find four or five apps in the managed Play Store, sign each of them in with an address typed on a tiny keyboard, wait for an SMS code that arrives on the employee's own phone, drag icons to the home screen and change the settings nobody remembers until the battery is flat by noon.

None of that is hard. It is slow, it is the same on every phone, and it is easy to get slightly wrong: the fifth phone of the day gets a different treatment from the first, and nobody can say afterwards which apps were actually signed in.

AndroidContextDeploy is the answer to that. The phone sits on the desk, plugged into a computer and **mirrored** in a window. The tool applies the settings, walks the Enrollment screens it knows, types the credentials wherever a sign-in field appears, waits patiently on the screens it must not touch, and ends with a **Checklist** of what was done and what is still left by hand.

CAUTION

It does not take the technician out of the loop, and it is not meant to. Android Enterprise forbids some actions over ADB, and multi-factor authentication belongs to the employee. The tool does everything around those moments, and tells you clearly when one has arrived.

1.2 Installing and launching it

A **release** is the normal way for a technician: GitHub Actions builds one for every tagged version, and it carries everything, `adb` included. **From source** is for development, or for a Linux too old for the release. Either way, prepare the phone once as in section 2.2.

Windows Download `AndroidContextDeploy-v...-windows.zip` from the GitHub **Releases** page. Right-click → **Extract all** (never run it from inside the zip), then double-click `AndroidContextDeploy.exe`. If SmartScreen warns about an unsigned app: **More info** → **Run anyway**. A phone that never appears needs its manufacturer's USB driver.

Linux Download `AndroidContextDeploy-v...-linux.tar.gz`, then in a terminal:
`tar -xzf AndroidContextDeploy-v*-linux.tar.gz`
`cd AndroidContextDeploy && ./AndroidContextDeploy`

From source Python 3.11+ with Tk, a virtual environment, `pip install -r requirements.txt -e .`, `python main.py` — below.

`deploy.json`, `banner.txt` and `locales/` sit next to the executable (at the root of the clone from source) and can be edited without rebuilding.

1.2.1 Linux: letting adb reach the phone

The release brings its own `adb`, but Linux only lets a normal user open a USB phone when **udev rules** allow it. Without them the device card stays on `no permissions`, however often the phone says **Allow**. Install the rules once, then unplug and plug the phone back:

Debian, Ubuntu, Mint `sudo apt install android-sdk-platform-tools-common`, then
`sudo usermod -aG plugdev $USER`, and log out and back in

```
Fedora          sudo dnf install android-tools
```

```
Arch, Manjaro, CachyOS  sudo pacman -S android-udev
```

The Linux release is built on Ubuntu 24.04 and needs glibc 2.39 or newer (Ubuntu 24.04, Debian 13, Fedora 40 and later); on an older system, run it from source. Under Wayland it runs through XWayland, like every Tk application.

1.2.2 From source

Install Python 3.11 or later **with Tk**:

```
Windows        The python.org installer — Tk is included.
```

```
Debian, Ubuntu  sudo apt install python3 python3-venv python3-tk
```

```
Fedora          sudo dnf install python3 python3-tkinter
```

```
Arch            sudo pacman -S python tk
```

```
git clone https://github.com/DireDoch/AndroidContextDeploy.git
cd AndroidContextDeploy
python3 -m venv .venv
source .venv/bin/activate # fish: activate.fish · Windows: .venv\Scripts\activate
pip install -r requirements.txt -e .
python main.py            # or: androidcontextdeploy
```

1.2.3 Options

Every form accepts two options:

```
--lang fr Force the UI Language. Without it: language in deploy.json, then the system locale.
```

```
--diag Capture every screen and every adb command to diag/, password and phone number masked. Chapter 7 explains what it is for.
```

1.3 What this manual covers

- **Chapter 2** — running the tool, and every feature it offers.
- **Chapter 3** — what it deliberately leaves to you.
- **Chapter 4** — how the project is put together, in diagrams.
- **Chapter 5** — the technologies it stands on, explained simply.
- **Chapter 6** — Enrollment, the hardest part, in depth.
- **Chapter 7** — extending it: an app, a setting, a language, a screen, a Module.
- **Chapter 8** — reading the Checklist and the console when something goes wrong.

Quick start and every feature

2.1 Before you run it

Computer	Windows 10/11 or a Linux desktop, one free USB port.
Phone	Android, with USB debugging on and this computer allowed.
Intune	A tenant where the apps of the Catalog are assigned to the employee. The tool never talks to Intune; the phone does.
The manifest	<code>deploy.json</code> names your Organization, apps and settings. The shipped file is a generic Microsoft example for “Example Corp”.
The employee	Nearby, with their own phone, for MFA.

2.2 Connecting the phone

The Mirror panel shows the three steps as long as no phone is plugged in:

1. **Settings** → **About phone** → **Software information**: tap **Build number** seven times. Developer options appear.
2. **Settings** → **Developer options**: turn on **USB debugging**.
3. Plug the cable in and tap **Allow** on the phone.

Within four seconds the device card turns green, the model, serial and battery appear, and the phone’s screen shows up live in the Mirror.

2.3 The window at a glance

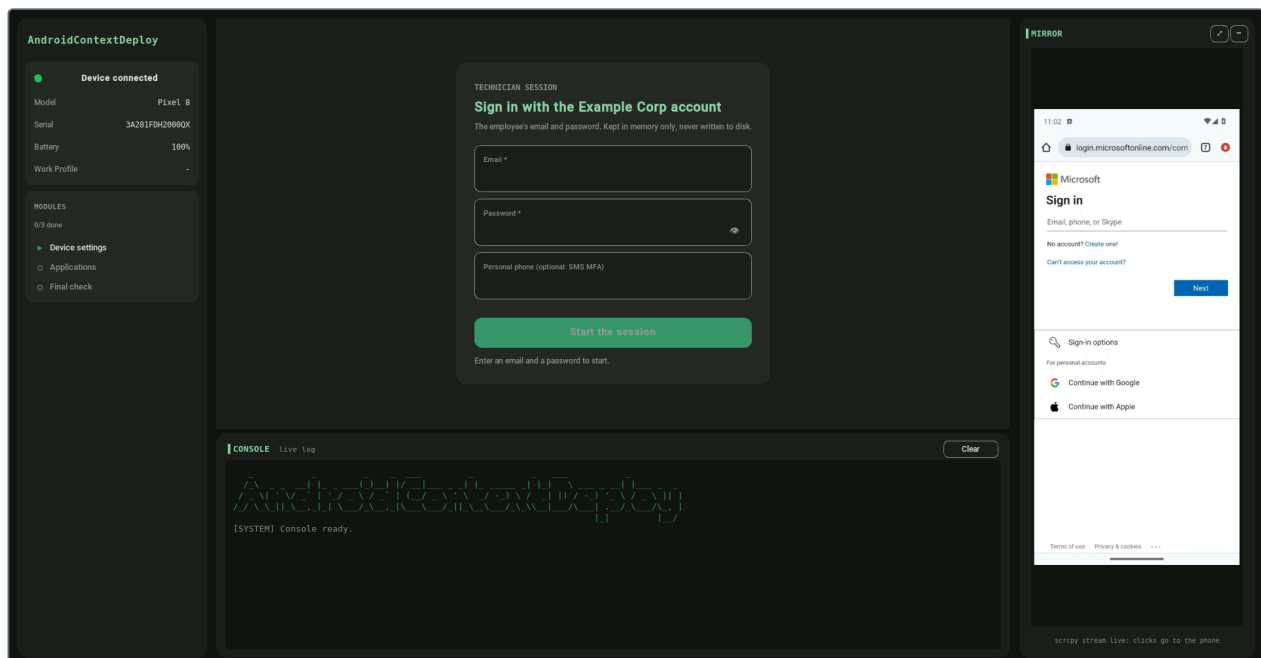


Figure 1: The window before a Session: device card and Modules on the left, the Session form and the console in the centre, the Mirror on the right.

Left rail Banner, device card (state, model, serial, battery, Work Profile user), Session summary, and the Modules with their progress.

Centre The Session form, then one card per Module, then the Checklist. Below: the console.

Right The Mirror. Clicks and drags on it are touches on the phone.

2.4 Starting a Session

Figure 2: The Session form.

Type the employee's **work email** and **password**. The **personal phone** is optional: when given, Enrollment types it on the SMS MFA screen; when not, that screen becomes a Manual Action.

These three values live **in memory only**. They are never written to disk, never logged, and they are gone when the window closes or a new Session starts.

The title names the Organization from `deploy.json`, so a technician can see at a glance which manifest is loaded.

2.5 The Modules

A Session runs three **Modules** in order. Each has its own card with a run button; the sidebar ticks each one off, green when everything went well, orange when a Result needs a look.

2.5.1 Device settings

Applies every entry of `device_settings` with `settings` put, then **reads each one back**. A setting only counts as OK when the phone reports the expected value: some manufacturers silently ignore a key, and the Checklist should say so rather than pretend.

```
[22:10:59] [OK] Screen timeout: 10 minutes: confirmed on the phone.
[22:10:59] [OK] Manual brightness: confirmed on the phone.
[22:10:59] [WARN] Brightness level: system/screen_brightness: phone reports 102, expected 160
```

2.5.2 Applications

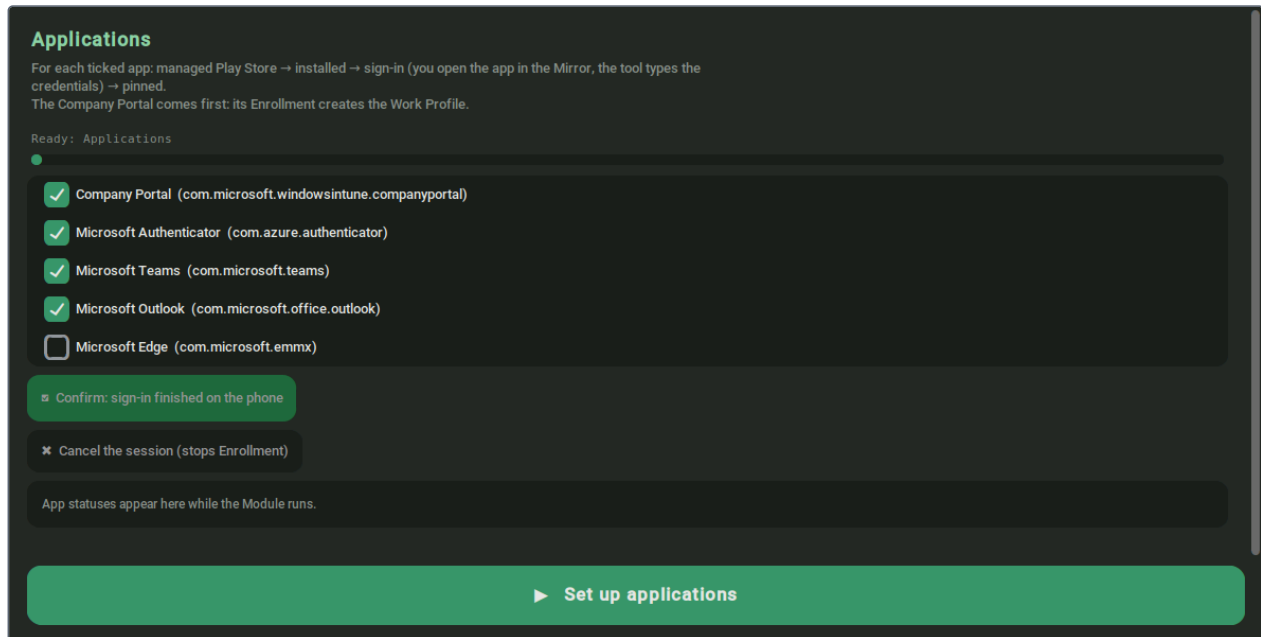


Figure 3: The Applications card: tick the apps for this phone. The order is the order of the Catalog.

For each ticked app, four steps:

1. **Store.** Its page opens in the managed Play Store, unless Intune already pushed it.
2. **Installed.** The tool waits, up to five minutes, until the app exists in the Work Profile.
3. **Sign-in.** The Company Portal goes through Enrollment (chapter 6); every other app goes through the Detection Loop: it reads the screen, types the email and password into the right fields, taps **Next** or **Add account**, waits for MFA.
4. **Pinned.** A shortcut goes on the home screen.

The first app is always the Company Portal: its Enrollment creates the Work Profile the others live in.

2.5.3 Manual Action, Confirm and Cancel

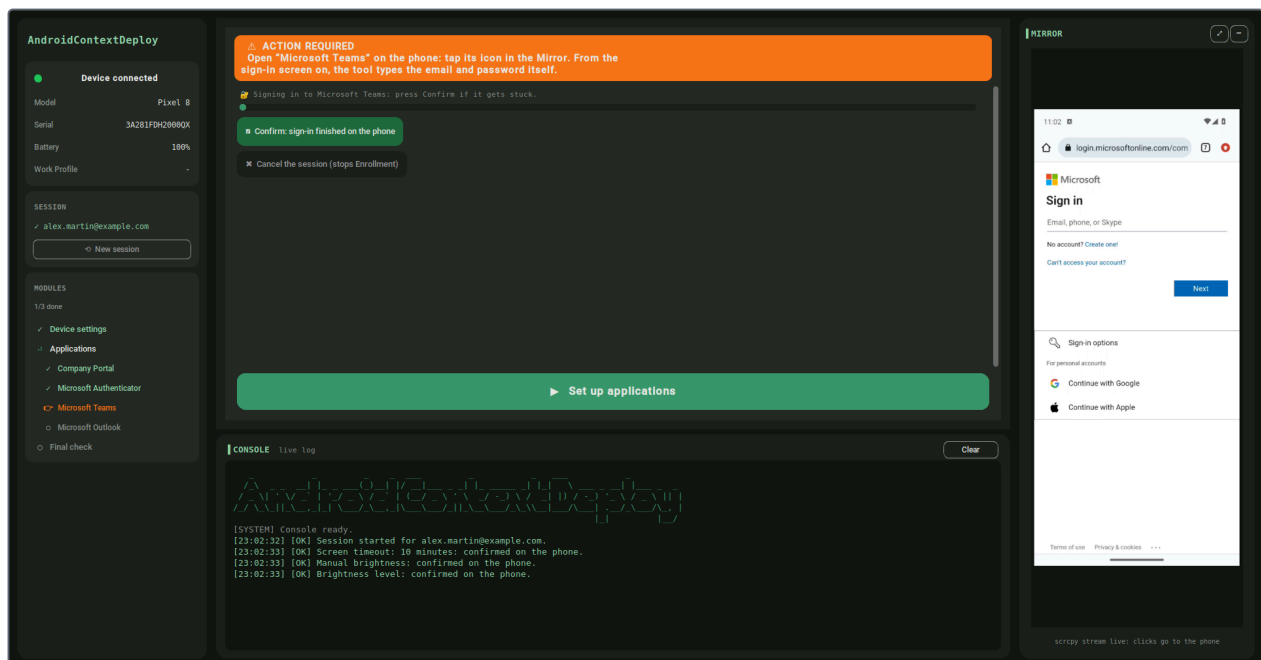


Figure 4: Mid-run: Teams needs to be opened by hand. The banner says so, the sidebar marks it orange, and Confirm is available.

Android Enterprise does not let ADB open an app inside the Work Profile. When that happens the card shows an orange **ACTION REQUIRED** banner naming the app: tap its icon in the Mirror. The tool is already watching; as soon as a sign-in field appears it types into it.

Confirm ends a sign-in loop that is stuck on a screen the tool does not know. **Cancel the session** is the only way to stop Enrollment (chapter 6).

2.5.4 Injection

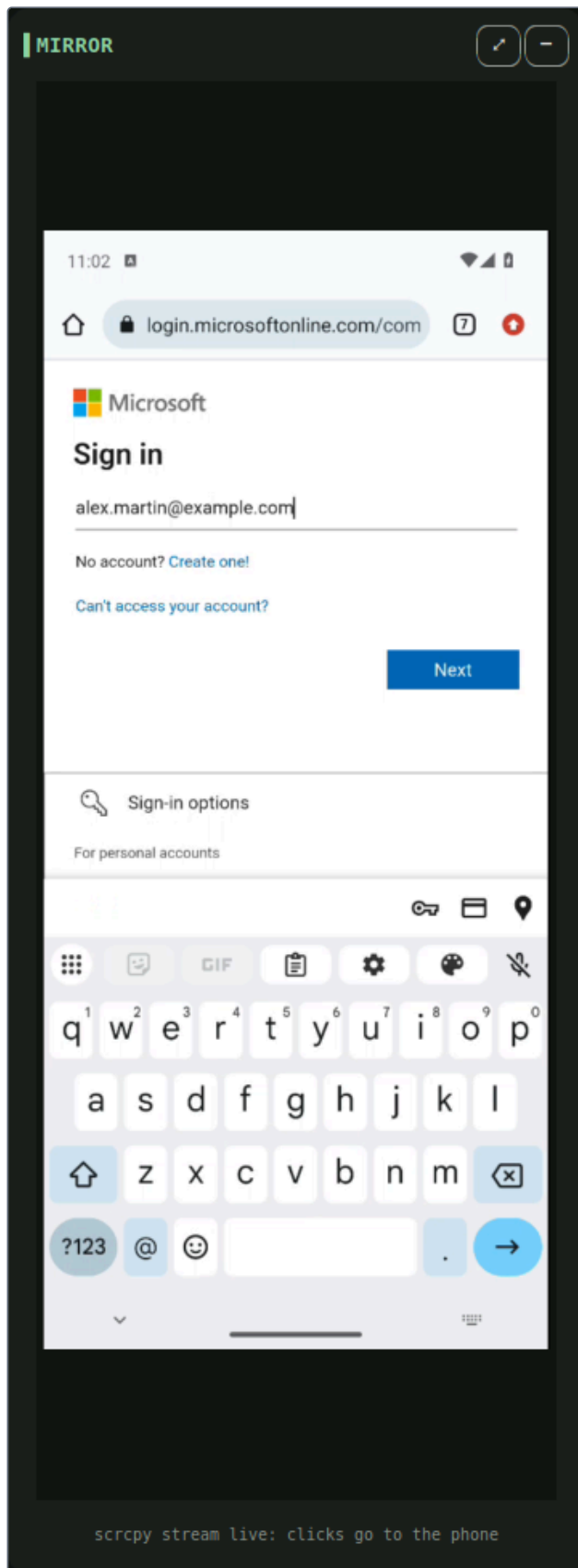


Figure 5: A real Microsoft sign-in page on the phone: the tool found the email field and typed into it.

Injection is the tool typing a credential into the field on the phone's screen. It only happens once the Detection Loop has **recognised** the field — by its id (`i0116` is Microsoft's email box), its hint, or its place on a sign-in screen — and seen that it is empty.

Two rules are never broken:

- the **password** is never logged, and neither is the phone number; both reach the phone on `adb shell`'s standard input, never on a command line;
- a value with accents is never typed (`adb` types ASCII only): the tool asks you to type it in the Mirror, and the Checklist notes it;
- a **one-time code field** is never typed into, even though its hint mentions "phone": that code belongs to the employee.

```
[INFO] Microsoft Teams: email field found.  
      Typing alex.martin@example.com.  
[OK] Email typed into Microsoft Teams.
```

2.5.5 Final check and the Checklist

The last Module reads the phone once more: is the Work Profile there, are the ticked apps still installed. Then the Checklist appears — every Result from every Module, in the order they ran.

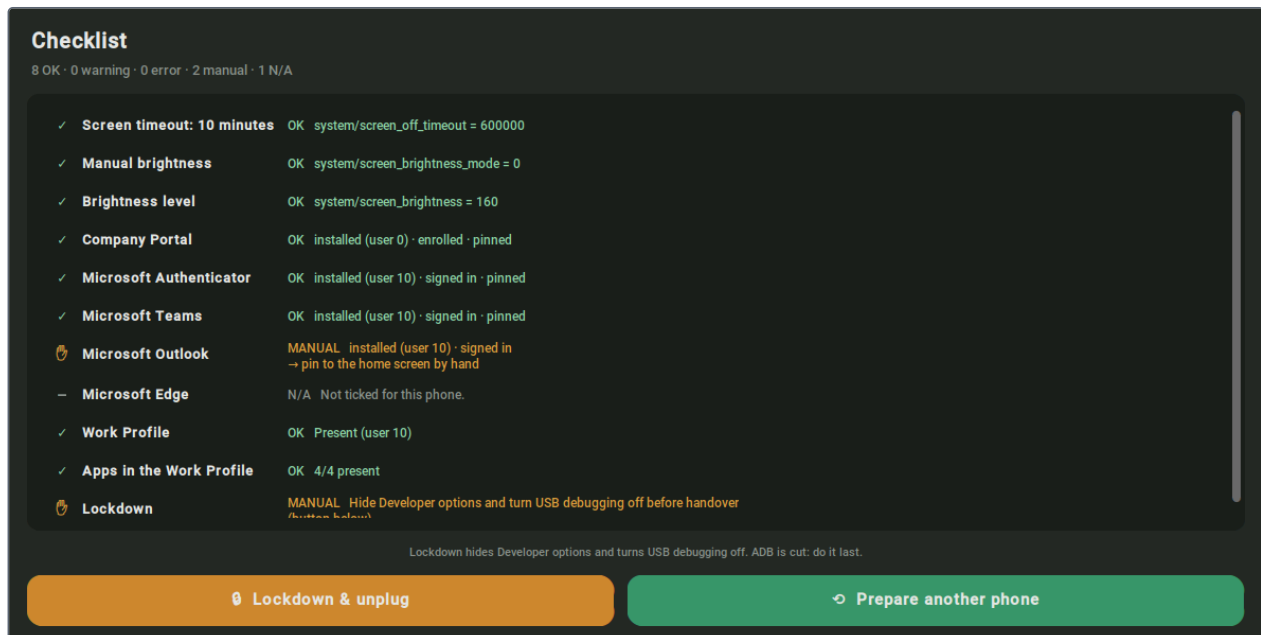


Figure 6: The Checklist. MANUAL rows are what is left to do by hand.

OK	Done and verified on the phone.
WARNING	Something should have worked and could not be verified. The line under it names the fix.
ERROR	Something failed that blocks handover (no Work Profile, Enrollment cancelled).
MANUAL	Yours to do: a pin, a sign-in the tool gave up on, Lockdown.
N/A	Does not apply to this phone (an app not ticked). Its absence is correct.

The same Checklist is printed in the console, so it can be copied into a ticket.

2.5.6 Lockdown

The **Lockdown & unplug** button hides Developer options, reads that back, and turns USB debugging off — in that order, because the last command cuts ADB for good. It asks for confirmation, stops the Mirror first, and turns the Lockdown row of the Checklist to OK. It is never run automatically.

2.5.7 The Mirror

The Mirror is the phone's screen at up to 30 frames per second, drawn at its real aspect ratio, and it is **touch-enabled**: a click is a tap, a drag is a swipe, and both land on the exact point under the mouse pointer — whatever the window size, the Mirror size or the display scaling (100 %, 125 %, 150 %...). Every Manual Action can be done from the computer without picking the phone up.

Only touches are sent: not the computer keyboard, not the mouse wheel, not the phone's hardware keys. Type with the phone's on-screen keyboard, in the Mirror.

The two buttons in its corner make it wider or fold it away. If it cannot start, the rest of the tool works exactly the same; the console says why.

2.5.8 The console

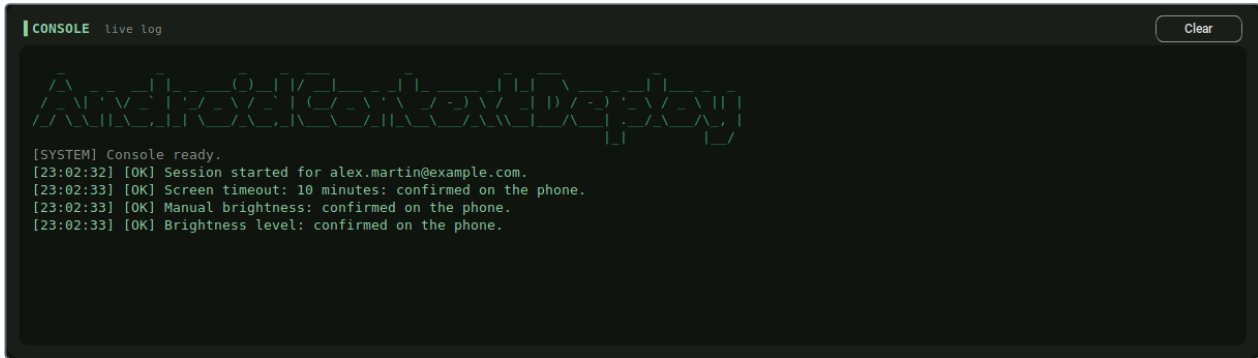


Figure 7: The console: every action and its result, coloured by level.

INFO is what the tool is doing, **OK** what it verified, **WARN** what needs a look, **ERROR** what failed. **Clear** empties it; the Checklist at the end repeats everything that matters.

2.5.9 Diagnostic capture

Started with `--diag`, the tool writes every screen it sees to `diag/session_<date>/`: the raw XML, a PNG, and a line explaining what it thought the screen was and what it was about to do, plus a log of every `adb` command. The password and phone number are masked before anything is written.

CAUTION

The PNGs can still show personal data **in pixels**. `diag/` never leaves the computer on its own; check it before sharing any part of it. Share the XML, not the PNGs.

2.5.10 Languages and branding

The window follows the **UI Language**: English and French ship, and any `locales/<code>.json` next to the tool becomes available. The phone's own screens are a different matter — see chapter 7.

The banner is `banner.txt`: paste your own ASCII art, or delete the file and the plain name is shown.

What stays manual

The tool automates everything ADB can do reliably. What remains is either forbidden to ADB by Android, or a secret only the employee has.

Step	Why	Who
Device settings	<code>settings put</code> , read back.	tool
Opening the store page	An intent on the managed Play Store.	tool
Typing email and password	Injection into a recognised field.	tool
Typing a value with accents	<code>adb</code> types ASCII only.	you
Enrollment screens it knows	Continue, Skip, Accept, “Example Corp device”, Phone method...	tool
Pinning to the home screen	A launcher broadcast. Some launchers refuse it.	tool, or you
Opening a Work Profile app	Android Enterprise refuses it to ADB (<code>SecurityException</code>).	you
Approving MFA	It happens on the employee’s authenticator.	employee
Typing the SMS code	It arrives on the employee’s phone.	employee
A screen the tool does not know	It waits rather than guess. Act in the Mirror.	you
Lockdown	Irreversible at the desk: a button, never a step.	you

Two words keep this clear in the code and in this manual:

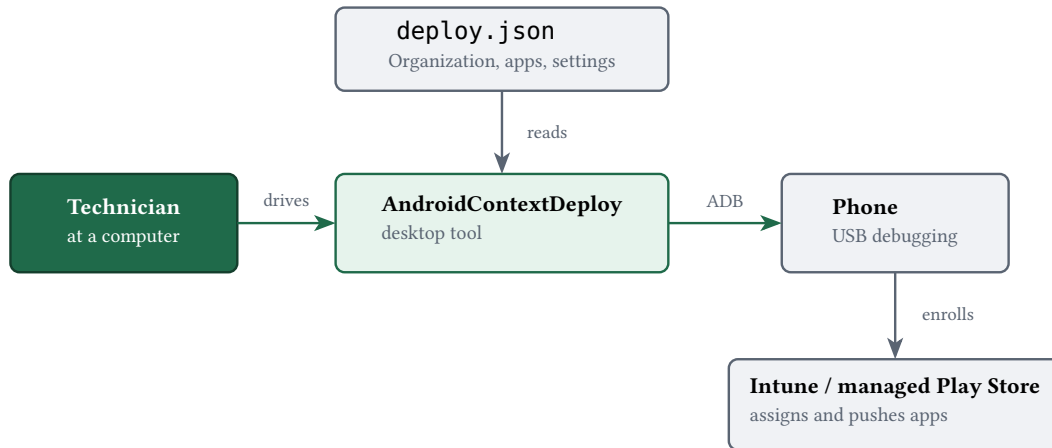
- a **Manual Action** is something to do **now**, announced by the orange banner while the tool waits;
- a **Manual Step** is a `MANUAL` row of the Checklist: something left **after** the run.

TIP

Thanks to Microsoft single sign-on, usually only the Company Portal asks for a real sign-in; the apps after it reuse the session. The Detection Loop simply finds nothing to type and moves on.

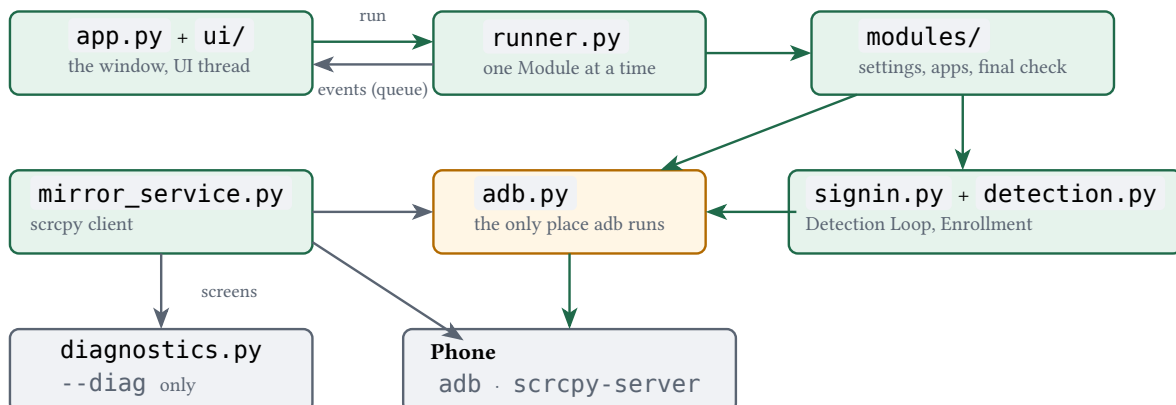
How the project is put together

4.1 Context — who talks to whom



The tool never talks to Intune. It remote-controls the phone, and the phone enrolls itself. That is why an app that is not **assigned** in Intune can never be fixed from the tool's side.

4.2 Containers — the pieces inside



Three rules keep the pieces apart, and they are what make the project testable:

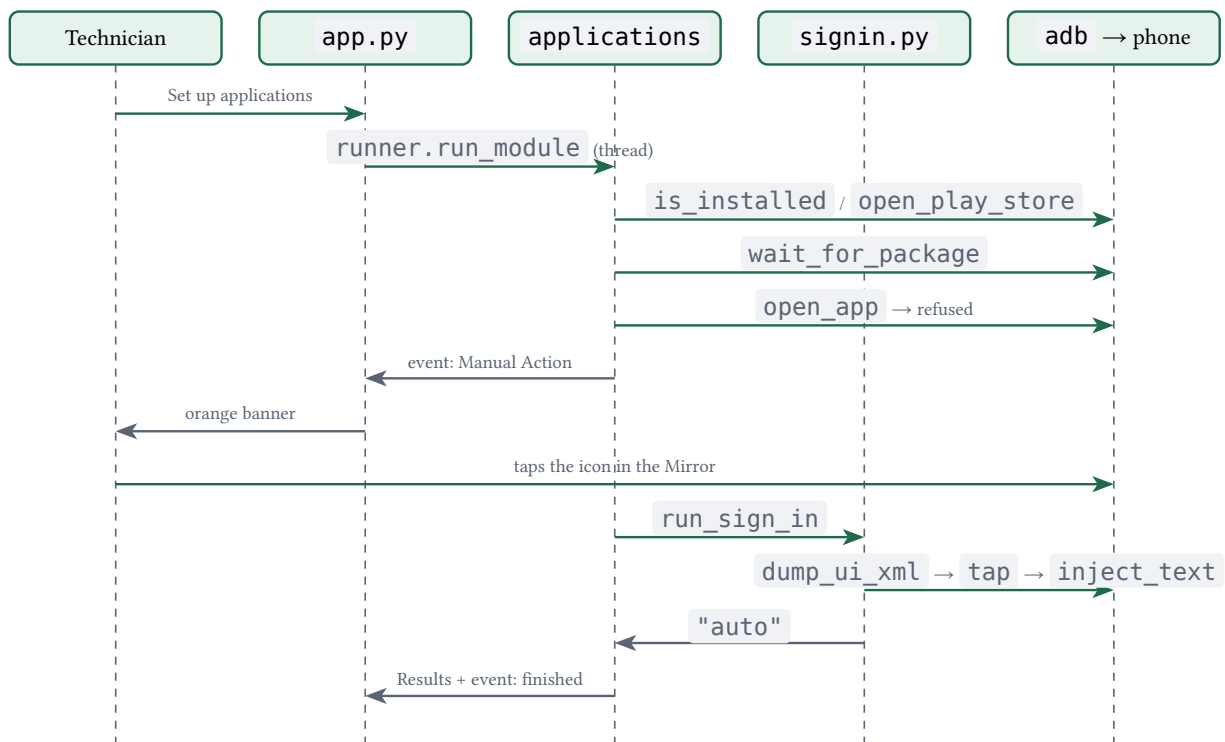
- **The UI thread never calls adb.** Modules run in the runner's thread and talk back through a queue that `app.py` drains every 150 ms.
- **Only `adb.py` runs adb.** Tests replace it with a fake phone; `--diag` hooks the single `_run` method and sees every command.
- **`detection.py` is pure parsing.** It gets XML, it returns what is on the screen. No phone is needed to test it.

4.3 The folder tree

<code>deploy.json</code>	the manifest: Organization, device_settings, catalog
<code>banner.txt</code>	the startup banner
<code>locales/en.json</code>	UI Language strings (fr.json, ...)
<code>main.py</code>	entry point: <code>--diag</code> , <code>--lang</code>
<code>pyproject.toml</code>	package metadata; ruff, mypy and pytest settings
<code>requirements.txt</code>	exact versions for CI and releases (-dev: plus tools)

build.py	PyInstaller bundle and archive
srcpy_server/	srcpy-server jar for the Mirror (Apache-2.0)
src/androidcontextdeploy/	
app.py	the window: Session, queues, wiring, Lockdown
runner.py	runs one Module off the UI thread, keeps every Result
modules/	__init__.py (MODULES, Context), device_settings.py,
	applications.py, final_check.py
adb.py	AdbService, DeviceMonitor
detection.py	ScreenDetector and the keyword tables
signin.py	SignInDriver: Detection Loop and Enrollment
mirror_service.py	srcpy client: H.264 in, touches out
diagnostics.py	DiagnosticRecorder, DiagnosticPoller (--diag)
manifest.py	deploy.json: load, validate, app_root()
i18n.py	t("key")
models.py	SessionConfig, DeviceInfo, Result, UiField, ScreenAnalysis
ui/	passive panels: rail, sidebar, form, cards, console, mirror
tests/	pytest; a fake phone and recorded screens
docs/	manual.typ, images/, adr/

4.4 Sequence — one app through the Applications Module



The technologies, explained simply

5.1 ADB — the cable that talks

Android Debug Bridge is Google's tool for talking to an Android phone from a computer. With USB debugging on, `adb shell` runs a command **on the phone** as a limited user. Almost everything this tool does is one of a handful of commands:

<code>settings put system screen_off_timeout 600000</code>	Change an Android setting.
<code>pm list packages --user 10</code>	List the apps of user 10 — the Work Profile.
<code>am start -a VIEW -d market://details?id=...</code>	Open an app's page in the Play Store.
<code>uiautomator dump</code>	Write the current screen's UI tree as XML.
<code>input tap 540 596 / input text '...'</code>	Touch the screen, type text.
<code>settings put global adb_enabled 0</code>	Turn USB debugging off (Lockdown).

In Python it is simply a subprocess: `AdbService._run` builds the command, runs it with a timeout, and returns `(ok, stdout, stderr)`. The `adb` binary itself comes from the `adbutils` package, so a release needs nothing installed.

5.2 uiautomator — reading a screen without looking at it

Android keeps, for accessibility, a tree of everything on screen. `uiautomator dump` writes it out. A Microsoft sign-in page looks like this, trimmed:

```
<node class="android.widget.TextView" text="Sign in" bounds="[80,400][1000,480]"/>
<node class="android.widget.EditText" resource-id="i0116" text=""
  focused="true" password="false" bounds="[40,540][1040,652]"/>
<node class="android.widget.Button" text="Next" clickable="true"
  bounds="[780,800][1040,880]"/>
```

`ScreenDetector` reads that tree and answers questions: is there an empty email field, and where is its centre? Is this an MFA screen? Which button leads on? It works by **keywords**: short lowercase fragments such as `"sign in"`, `"se connector"`, `"i0116"`. That is why a screen in a new language, or a new manufacturer's wording, is fixed by adding fragments — never by guessing coordinates.

5.3 Android Enterprise and the Work Profile

A company phone keeps company apps in a separate, managed space: the **Work Profile**, a second Android user (often `user 10`) whose apps carry a briefcase icon. The **Company Portal** creates it during Enrollment, and **Intune** then decides which apps it receives through the **managed Play Store**.

For this tool, two consequences matter. Every app command needs `--user 10`, found by looking for the managed-profile flag in `pm list users`. And ADB is **not allowed** to launch an app inside that profile — hence the Manual Action.

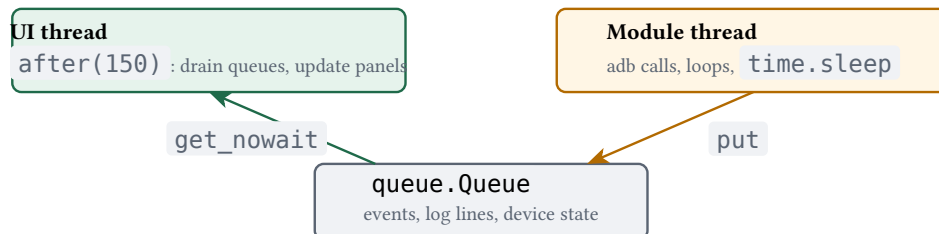
5.4 scrcpy, H.264 and PyAV — the Mirror

scrcpy is an open-source screen mirroring project. Its small server, a `.jar`, is pushed to `/data/local/tmp` and started with `app_process`: nothing is installed. It sends the screen as an **H.264** video stream on one socket and accepts touch events on another.

PyAV (Python bindings to FFmpeg) decodes that stream into frames; **numpy** holds them; **Pillow** turns the latest one into an image the window can draw, about 25 times a second. A click becomes a 32-byte touch message in `scrcpy`'s control protocol.

5.5 CustomTkinter, threads and queues — the window

Tkinter is the GUI toolkit that ships with Python; **CustomTkinter** gives it a modern look. Tk has one hard rule: only the thread that created the window may touch it. Anything slow — every adb call — therefore runs in another thread.

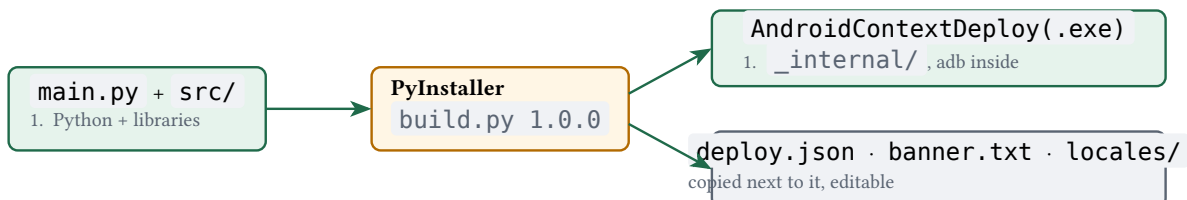


A Module never knows the window exists: it calls `ctx.runner.emit_progress(...)`, which puts a small dictionary on a queue. The UI thread picks it up on its next tick. That separation is also what lets tests run a Module with no window at all.

5.6 JSON — the manifest and the languages

`deploy.json` and `locales/*.json` are plain JSON: data, never code. The manifest is **validated** when the tool starts — a missing `package_id`, an unknown `namespace`, an Enrollment app that is not first — and a mistake is shown with the exact key to fix before anything touches a phone. Strings go through `t("key", app="Teams")`, which looks the key up in the current language, falls back to English, and fills the `{app}` placeholder.

5.7 PyInstaller — from Python to an executable



PyInstaller collects a Python program, the interpreter and every library into a folder with a launcher. The build is **onedir** (a folder, not a single file) so it starts instantly. GitHub Actions runs `build.py` on Windows and on Ubuntu for every `v*` tag and attaches both archives to the release.

5.8 pytest and fakes — tests without a phone

A test that needs a real phone is a test nobody runs. The tests replace `AdbService` with a **fake**: a small class with the same method names that returns recorded screens and remembers what it was asked to tap and type.

```

outcome, adb, log = _sign_in([DUMP_EMAIL_WEBVIEW, DUMP_EMAIL_FILLED,
                              DUMP_PASSWORD, DUMP_PASSWORD_FILLED,
                              DUMP_HOME, DUMP_HOME, DUMP_HOME])
assert [a[1] for a in adb.actions if a[0] == "text"] == [EMAIL, PASSWORD]
assert "S3cret" not in log          # the password is never logged
  
```

`python -m pytest` runs the whole suite in a few seconds, on Linux or Windows. CI adds three checks on every push: `ruff` (style and common bugs), `mypy` (types) and coverage of at least 80 %, measured with the

window tests running under Xvfb — one of them clicks the Mirror and checks that the touch lands on the pixel under the pointer.

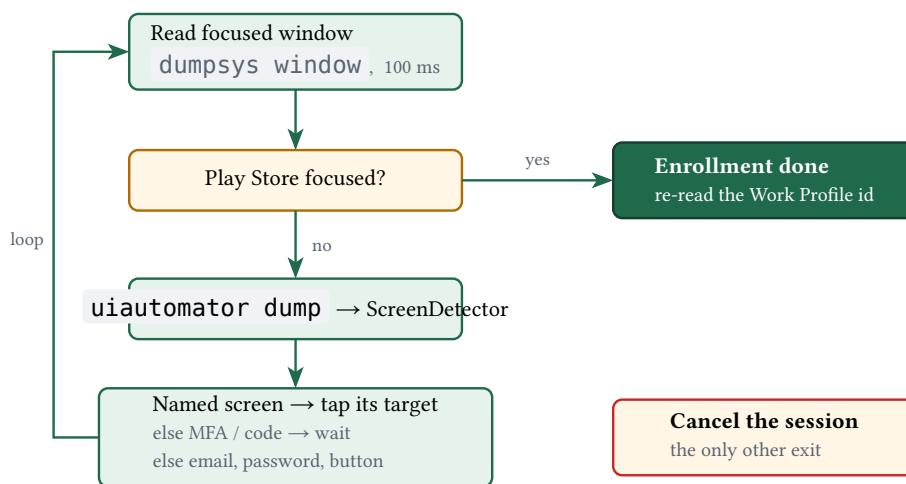
Enrollment in depth

6.1 What Enrollment is

Enrollment is the Company Portal’s first run: sign-in, the “set up access” screens, admin activation, policies, MFA setup, the terms, the ownership choice. When it finishes, the phone has a Work Profile and Intune starts pushing apps. Everything after it depends on it, which is why it is always the first entry of the Catalog.

6.2 The gate: the managed Play Store, never a timer

The generic Detection Loop decides a sign-in is over when the sign-in screen has gone. For Enrollment that is badly wrong: the sign-in screen disappears long before the policies, the MFA and the terms. So Enrollment has exactly **one** exit:



If the employee walks away mid-MFA, the tool waits — for an hour if it has to. That is deliberate (ADR-0004): a timeout would stop good Enrollments during a slow policy push, and a half-finished Enrollment leaves no Work Profile and every following app on the wrong user. **Confirm** does not bypass the gate; only **Cancel the session** does, and the Checklist records it as an ERROR.

6.3 The named screens

Before the generic mechanics, `ScreenDetector` checks for the Enrollment screens it knows, most specific first. Recognising one also stops the generic mechanics from tapping the wrong button on it.

Screen (key)	What the tool does
Ownership (<code>ownership</code>)	Taps the option containing the Organization from <code>deploy.json</code> (“Example Corp device”), never “Personal”, then Finish.
Method dialog (<code>method_dialog</code>)	Taps Phone , then Confirm .
Phone entry (<code>phone_entry</code>)	Types the Session’s personal phone, ticks “Text me a code”, taps Next. No number: Manual Action.
Authenticator (<code>authenticator_other_method</code>)	Taps the “set up a different method” link , not the big Next button.
Terms (<code>terms</code>)	Accept.
Access setup (<code>access_setup</code>)	Continue — never “Sign out”, which sits right next to it.

Finish account setup (`skip_setup`)

Skip.

A screen that matches nothing is logged with the buttons it shows, and the tool waits for the next screen change. It never taps something it does not recognise.

6.4 Why the Organization matters

The ownership screen is the one place where a wrong tap is expensive: choosing “Personal” enrolls the phone as a personal device, with different policies. The detector only taps an option containing the `organization` string from `deploy.json`. A manifest that says “Example Corp” on another company’s phone taps nothing, and the technician chooses in the Mirror.

Extending it

7.1 First: does it need code at all?

Most of what a company needs is data, and data lives in `deploy.json`. An entry gets a sidebar row, a progress line, console lines and a Checklist row with a remediation — for free.

An app. Find its id in its Play Store address (`.../details?id=com.Slack`):

```
{ "name": "Slack", "package_id": "com.Slack", "sign_in": true, "pin": true }
```

Add `"activity": "com.Slack/.MainActivity"` only if the tool cannot open or pin the app: the phone's own answer is always tried first.

An Android setting. Any key `settings get` can read:

```
{ "name": "Stay awake while charging", "namespace": "global",  
  "key": "stay_on_while_plugged_in", "value": "3" }
```

Find a key by changing the setting by hand and comparing `adb shell settings list system` (or `secure, global`) before and after.

7.2 Adding a UI Language

1. Copy `locales/en.json` to `locales/es.json`.
2. Translate the **values**. Keep the keys, and keep every `{placeholder}` as it is.
3. Run `python -m pytest: tests/test_i18n.py` fails on a missing key or a changed placeholder.
4. Start with `--lang es`, or set `"language": "es"` in `deploy.json`.

In a release, drop the file into `locales/` next to the executable. No rebuild.

7.3 Teaching it a new screen

This is the contribution the tool needs most, because every manufacturer, Android version, Phone Language and Intune configuration words its screens a little differently.

1. **Capture.** Run with `--diag` and reproduce until the tool is stuck. The console names the screen as **UNRECOGNISED** and lists its buttons.
2. **Find it.** In `diag/session_.../`, `trace.jsonl` numbers the screens; open the matching `NNN.xml`. The password and phone number are already masked; replace any other personal text (names, the email) with Example Corp values.
3. **Lock it in a test.** Paste the XML into `tests/test_detection.py` as a `DUMP_...` constant and assert what the detector should see — it fails.
4. **Teach it.** Add the short fragments to the right keyword table in `detection.py` (for instance `_ACCESS_SETUP_KEYWORDS` for a Spanish “configurar el acceso”). The test turns green.
5. **Send it.** Open a pull request with the fixture and the keywords. Never the PNG.

```
DUMP_ACCESS_SETUP_ES = _wrap(  
    '<node class="android.widget.TextView" text="Configurar el acceso a Example  
Corp" .../>'  
    '<node class="android.widget.Button" text="CONTINUAR" clickable="true" '  
    ' bounds="[560,1700][1040,1800]"/>')'
```

```
def test_access_setup_in_spanish() -> None:
    s = ScreenDetector("Example Corp").analyze(DUMP_ACCESS_SETUP_ES)
    assert s.named_screen == "access_setup"
```

NOTE

Keywords are short **fragments**, lowercase, matched anywhere in the screen's text. Prefer "configurar el acceso" to the whole sentence: a comma or an apostrophe changed by an update should not break recognition.

7.4 Adding a Module

Write a Module only when `deploy.json` cannot express the need — reading battery health, checking the Android security patch level, setting a wallpaper.

7.4.1 The four steps

1. Create `src/androidcontextdeploy/modules/my_module.py` from the template below.
2. Append it to `MODULES` in `modules/__init__.py`, where it should run.
3. Add `module.my_module.title`, `.description` and `.run`, and your result strings, to **both** `locales/en.json` and `locales/fr.json`.
4. Add a test with a fake phone, as in `tests/test_modules.py`.

7.4.2 The template

```
"""Reports the Android security patch level, and warns when it is old."""
from __future__ import annotations

from datetime import date

from androidcontextdeploy.i18n import t
from androidcontextdeploy.models import Result

NAME = "security_patch"
MAX_AGE_DAYS = 120

def run(ctx) -> list[Result]:
    ctx.runner.emit_progress(0.5, t("progress.security_patch"))
    patch = ctx.adb.get_prop(ctx.device.serial, "ro.build.version.security_patch")
    if not patch:
        return [Result(t("module.security_patch.title"), "WARNING",
                        t("result.security_patch.unknown"))]
    age = (date.today() - date.fromisoformat(patch)).days
    if age > MAX_AGE_DAYS:
        return [Result(t("module.security_patch.title"), "WARNING",
                        t("result.security_patch.old", patch=patch, days=age),
                        t("remedy.security_patch"))]
    return [Result(t("module.security_patch.title"), "OK", patch)]
```

7.4.3 What the template is showing you

- **NAME** is the key for its strings and its sidebar row.

- `run(ctx)` gets everything through `ctx`: `ctx.adb` for the phone, `ctx.device.serial`, `ctx.manifest`, `ctx.apps`, `ctx.session`, `ctx.log`, `ctx.runner` for progress. It never imports the UI.
- **It returns Results**, at least one. A problem is a `WARNING` or `ERROR` Result with a `remedy`, not an exception: the run goes on and the Checklist shows it. An unexpected exception is still caught by the runner and becomes an `ERROR`.
- **It only calls methods of `AdbService`**. If the command you need is not there, add a method to `adb.py` — never call `subprocess` from a Module.

7.4.4 Testing it

```
def test_old_patch_is_a_warning() -> None:
    phone = FakePhone()
    phone.get_prop = lambda serial, prop: "2024-01-05"
    results = security_patch.run(_ctx(phone))
    assert results[0].kind == "WARNING" and results[0].remedy
```

7.5 Building a release

```
pip install -r requirements-dev.txt
python build.py 1.0.0
```

Or push a tag: `git tag v1.0.0 && git push --tags`. GitHub Actions builds the Windows zip and the Linux tarball and publishes them with `SHA256SUMS.txt` and a build provenance attestation, which anyone can check before running a download: `gh attestation verify <archive> -R DireDoch/AndroidContextDeploy`.

Troubleshooting

8.1 Reading the Checklist

Start with the `ERROR` rows, then `WARNING`, then `MANUAL`. Every row that needs something carries a line starting with an arrow: that is the fix, usually naming the `deploy.json` key or the Intune assignment.

```
[OK      ] Screen timeout: 10 minutes    system/screen_off_timeout = 600000
[WARNING] Microsoft Outlook              Not installed after 5 minutes
                                         -> Assign it to this user in Intune, or untick /
                                         remove catalog "Microsoft Outlook" in deploy.json.

[MANUAL ] Microsoft Teams                installed (user 10) · signed in
                                         -> pin to the home screen by hand

[ERROR  ] Work Profile                   No Work Profile on the phone
[N/A    ] Microsoft Edge                 Not ticked for this phone.
```

8.2 Common problems

Symptom	What to do
A <code>deploy.json</code> error at start	The message names the key and the line. Fix it; nothing ran.
No device detected	Check the cable (a charge-only cable carries no data), USB debugging, and the Allow prompt on the phone.
Device not ready (unauthorized)	Unplug, plug back, tap Allow . Tick “Always allow from this computer”.
adb unavailable	Another adb server may hold the port: close Android Studio or scrcpy, or point <code>ADB_COMMAND</code> at the same adb they use.
Mirror unavailable	See the console line. Everything else works without it.
An app never installs	It is not assigned in Intune for this user, or the Work Profile does not exist yet. Check the Work Profile row first.
The tool does not open an app	Expected in a Work Profile: follow the orange banner and tap it in the Mirror.
Stuck on a screen during Enrollment	It waits by design. Act in the Mirror. If the screen should be known, capture it with <code>--diag</code> (chapter 7).
Typed into the wrong field	Stop with Confirm or Cancel, fix by hand, and report the screen with its XML.
ACTION REQUIRED: type it yourself	The email, password or phone number has accents or other non-ASCII characters, which <code>adb</code> cannot type. Type it in the Mirror; the tool carries on when the screen changes.
Pinning always fails	Some launchers refuse shortcut broadcasts. It stays a Manual Step; set <code>"pin": false</code> to stop trying.
A setting always WARNs	The key is read-only or renamed on that manufacturer. Remove it from <code>device_settings</code> .

8.3 Getting more detail

The console holds every action; select and copy it into an issue. For a screen problem, run again with `--diag : trace.jsonl` says, screen by screen, what the detector believed and what the tool did, and `adb.log` has every command with its raw output. That is almost always enough to fix it without ever seeing the phone.